

C++ Essentials: Cheat Sheet

Stefanie Zollmann

August 26, 2025

This document covers the essential concepts of C++ relevant to this course, reframing the original lab material into a concise reference guide.

1 Why C++?

C++ is a powerful, general-purpose programming language. While complex, it offers significant advantages, especially in fields like Visual Computing.

- **Performance:** C++ is designed for systems programming, resulting in faster execution compared to languages like Java or C#. This is critical for real-time and computationally intensive tasks.
- **Seamless Integration:** Core libraries like OpenCV and OpenGL are built with C/C++ at their heart, making integration smooth and efficient.
- **Expressive Syntax:** Features like **operator overloading** allow for more intuitive code. For example, the matrix equation $\vec{u} = A\vec{v} + \frac{1}{2}\vec{w}$ can be written naturally:

```
1 u = A*v + 0.5*w;  
2
```

This is much cleaner than the more verbose method calls required in other languages:

```
1 u = A.multiply(v).add(w.multiply(0.5));  
2
```

- **Modern Abstractions:** Compared to C, C++ offers superior tools for data management, a stronger type system, and safer ways to handle resources, which helps prevent common errors.

2 Variables, Pointers, and References

You're likely familiar with standard variables and pointers from C.

```
1 int var = 3; // An integer variable initialized to 3.  
2 int* ptr = &var; // A pointer to an integer, pointing to 'var'.
```

C++ adds **references**, which act as an alias for an existing variable.

```
1 int& ref = var; // A reference to an integer, referring to 'var'.
```

References are similar to pointers but with crucial differences:

- **No Nulls:** A reference **must** be initialized to refer to an existing variable; it cannot be null or uninitialized.
- **Alias Behavior:** A reference is an alias, not a memory address. You don't need to dereference it. Modifying `ref` directly modifies `var`. For example, `*ptr = 5;` and `ref = 5;` both change `var` to 5.
- **No Arithmetic:** You cannot perform pointer arithmetic on references. `ref + 2` adds 2 to the value of `var`, whereas `ptr + 2` would move the pointer's memory address.

When accessing members of an object, use `.` for the object or a reference, and `->` for a pointer.

```

1 Object obj;
2 Object* ptr = &obj;
3 Object& ref = obj;
4
5 obj.method(); // Access with .
6 ptr->method(); // Access with ->
7 ref.method(); // Access with .

```

2.1 Memory Management

In C++, dynamic memory is managed with `new` and `delete`, which are type-safe alternatives to C's `malloc` and `free`.

```

1 // Allocate a single double and an array of 10 doubles.
2 double* dptr = new double;
3 double* darr = new double[10];
4 // ...
5 // Deallocate the memory. Note the [] for the array.
6 delete dptr;
7 delete [] darr;

```

Modern C++ strongly favors **smart pointers**, which automate memory management. They follow the **RAII (Resource Acquisition Is Initialization)** idiom, where a resource is freed when its controlling object (the smart pointer) goes out of scope.

```

1 #include <memory> // Required header for smart pointers
2
3 // A unique_ptr ensures there is only one owner of the memory.
4 std::unique_ptr<double> dptr(new double);
5
6 // When 'dptr' goes out of scope, the memory is automatically deleted.
7 // No 'delete' call is needed!

```

3 C++ Objects

A C++ class is typically defined in a header file (`.h`) and implemented in a source file (`.cpp`). Access modifiers (`public`, `protected`, `private`) work similarly to Java but apply to all subsequent members in a block.

3.1 Construction, Copying, and Destruction

C++ gives you fine-grained control over an object's life cycle through special member functions:

- **Constructor:** Initializes the object (`Object()`).
- **Copy Constructor:** Creates a new object as a copy of an existing one (`Object(const Object& obj);`).
- **Copy Assignment Operator:** Overwrites an existing object with a copy of another (`operator=`).
- **Destructor:** Cleans up resources when the object is destroyed (`~Object()`).

If your class manages resources like raw pointers, you must implement these correctly to avoid issues (the **Rule of Three/Five**).

3.2 Initializer Lists

The preferred way to initialize member variables in a constructor is with an **initializer list**. This ensures members are initialized *before* the constructor body runs.

```

1 class MyObject {
2 private:
3     int id_;
4     double value_;
5 public:
6     // Initializer list starts with ':'
7     MyObject(int id, double value) : id_(id), value_(value) {
8         // Constructor body can be empty or contain other logic.
9     }
10 };

```

4 Inheritance and Polymorphism

C++ supports object-oriented inheritance. A derived class inherits from a base class using the following syntax:

```
1 class Derived : public Object {
2     // ...
3 };
```

Here, `public` inheritance creates an "is-a" relationship.

4.1 The `virtual` Keyword

A common problem arises when you use a base class pointer to refer to a derived class object. To ensure the correct derived-class methods and destructor are called, declare the base class functions as `virtual`.

```
1 class Object {
2 public:
3     // Destructors should almost always be virtual in a base class.
4     virtual ~Object();
5
6     // A virtual function allows polymorphism.
7     virtual void whatAmI();
8 };
```

By making `~Object()` and `whatAmI()` virtual, the correct derived class methods will be called through a base class pointer or reference, ensuring correct polymorphic behavior and proper cleanup.

5 The Standard Template Library (STL)

The STL is a powerful collection of template classes and functions. **Templates** allow you to write code that works with any data type.

```
1 template <typename T>
2 const T& minimum(const T& a, const T& b) {
3     return a < b ? a : b;
4 }
```

5.1 The `std::vector` Class

The most common STL container is `std::vector`, a dynamic array.

```
1 #include <vector>
2
3 std::vector<int> numbers;    // Create an empty vector of integers.
4 numbers.push_back(10);      // Add an element to the end.
5 numbers.push_back(20);
6
7 // Use a range-based for loop to iterate.
8 for (const int& num : numbers) {
9     // ...
10 }
```

5.2 The `auto` Keyword

The `auto` keyword tells the compiler to automatically deduce the type of a variable from its initializer, making code cleaner.

```
1 // Instead of: std::vector<int>::const_iterator iter = numbers.begin();
2 auto iter = numbers.begin(); // Much simpler!
```

6 `const` Correctness

The `const` keyword is a promise that a value will not be modified. Using it properly is called **const-correctness** and is a cornerstone of good C++ code.

- **Constant variables:** `const double PI = 3.14159;`
- **Constant references:** Pass large objects by `const` reference to avoid copies and guarantee they won't be changed.

```
1 void printObject(const MyObject& obj);  
2
```

- **Constant member functions:** A member function declared `const` promises not to modify the object's state.

```
1 class MyObject {  
2 public:  
3     int getValue() const; // This method does not change the object.  
4 };  
5
```

A good rule of thumb is to use `const` whenever possible.